NO SQL DATABASES

MODULE 3 – STUDY NOTES

MapReduce and Key–Value Databases

Prepared for: Department of Artificial Intelligence and Machine Learning
Acharya Institute of Technology (AIT)

Based on the Module 3 portion (PDF pages 25–37) of the supplied NoSQL study material. Original instructional figures have been retained from the supplied textbook/study-material PDF.

MODULE 3 – CONTENTS

• MapReduce: concept, phases and workflow

• Basic Map and Reduce functions

• Partitioning and Combining

• Composing MapReduce calculations

• Two-stage MapReduce example

• Incremental MapReduce

• Key–Value databases: model and terminology

• Features and CRUD operations in Riak

• Scaling in Riak and suitable use cases

• When not to use Key–Value stores

• VTU-oriented probable questions

Learning outcomes

After studying this module, students should be able to explain the MapReduce programming model, trace the MapReduce workflow, distinguish partitioning from combining, describe multi-stage and incremental MapReduce, explain the key–value data model and its terminology, outline Riak CRUD operations and scaling, and identify suitable and unsuitable applications of key–value stores.

1. MapReduce

MapReduce is a programming model and processing framework for distributed computing, introduced by Google. It enables large datasets to be processed using a distributed algorithm across a cluster of machines. The framework primarily works in two phases: **Map** and **Reduce**.

ıce phase.

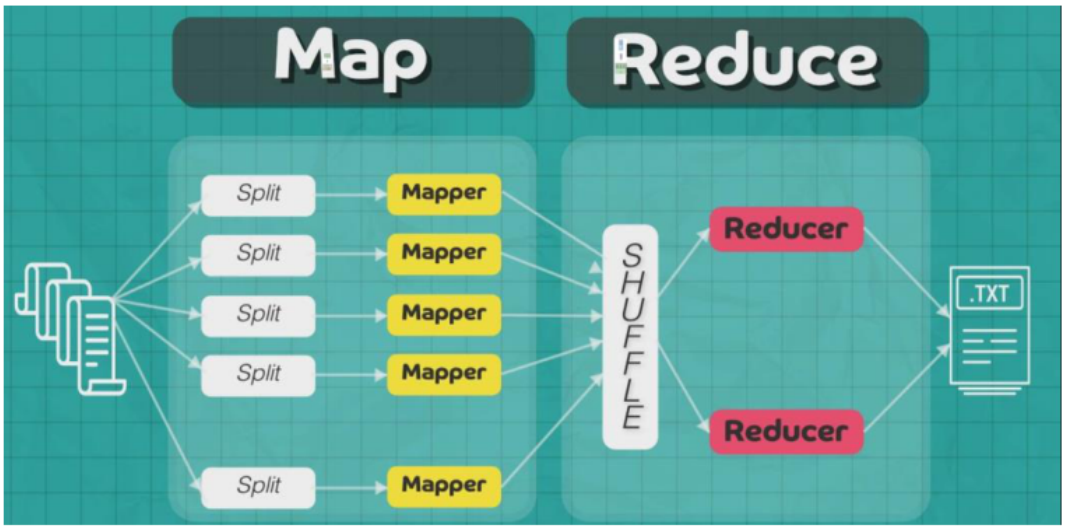


Figure 1. Original MapReduce workflow diagram from the supplied study material (PDF page 25).

1.1 Map function

The Map function takes an input key/value pair or input record and produces a set of intermediate key/value pairs. It is applied independently to input records and can therefore execute in parallel across multiple data chunks. In the order-processing example in the study material, an order is the input aggregate and the mapper emits key/value pairs for its line items. The product ID is used as the key, while quantity and price are carried as the associated values.

1.2 Reduce function

The Reduce function receives intermediate values associated with the same key and combines or aggregates them to produce the final result. It groups information by key and performs operations such as summation, counting or merging.

1.3 MapReduce workflow

Stage	Description
1. Splitting	Input data is divided into manageable chunks, often by a distributed file system such as HDFS.
2. Mapping	The Map function processes each chunk and emits intermediate key/value pairs.
3. Shuffling and sorting	Intermediate records are transferred, sorted and grouped by key so that values for the same key reach the same re
4. Reducing	The Reduce function processes each key and its grouped values to generate results.
5. Output	Final results are written to an output store such as HDFS or another distributed storage system.

1.4 Basic MapReduce

A MapReduce job begins with the Map stage. A mapper reads an input aggregate and emits key/value pairs. The framework then groups records by key. A reducer receives multiple values for a key and aggregates them into one result. The two original figures below illustrate the mapper reading database records and the reducer combining values with matching keys.

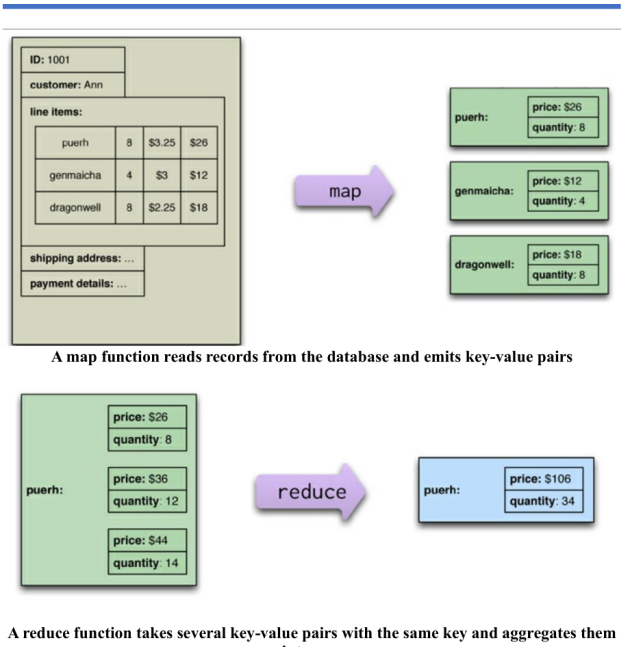


Figure 2. Original Map and Reduce examples (PDF page 26).

2. Partitioning and Combining

2.1 Partitioning

Partitioning distributes the intermediate key/value pairs generated by the Map phase among the available reducers. A partitioner decides which reducer receives each intermediate key. A common strategy uses a hash function on the key so that records are distributed across reducers. Good partitioning balances the workload and allows reducers to process different keys in parallel.

• The partitioner determines the reducer responsible for an intermediate key.

• All values belonging to the same key must be sent to the same reducer.

• A balanced distribution avoids overloading one reducer and improves parallel execution.

2.2 Combining

Combining is an optimization performed on mapper-side intermediate data before it is transferred over the network. A combiner acts like a mini-reducer and performs local aggregation. This reduces the amount of data shuffled across the network, lowering communication overhead and improving efficiency.

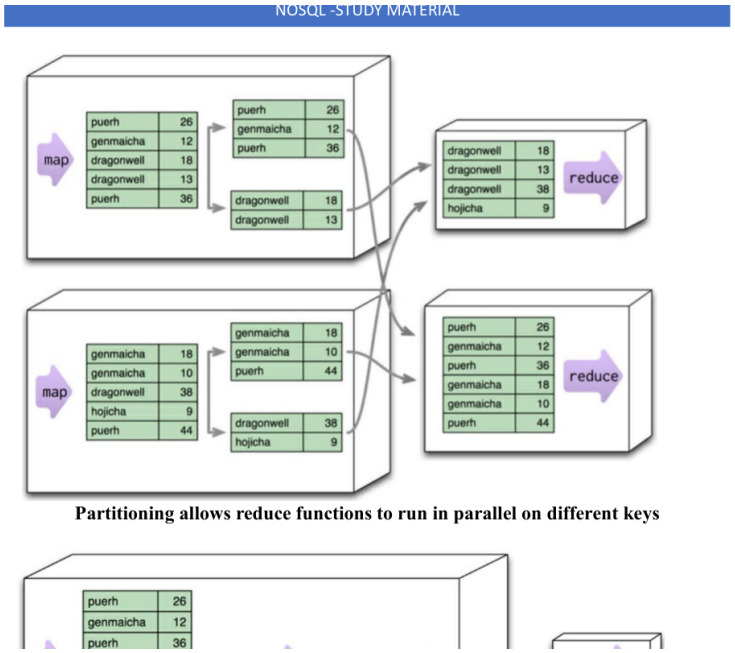


Figure 3. Original diagrams showing partitioning and combining (PDF page 27).

2.3 Combinable reducer

A combiner is essentially a reducer function. In some cases, the same function can be used for both combining and final reduction. For this to be valid, the reducer output must have the same form as its input; such a reducer is called a **combinable reducer**.

Important condition: Not every reduce function is combinable. The study material gives the example of counting how many unique customers ordered a particular tea. A local combiner cannot simply add local distinct-customer counts if the same customer may appear in more than one mapper partition; doing so can double-count customers.

3. Composing MapReduce Calculations

Some processing tasks require more than one MapReduce job. Composing MapReduce calculations means chaining jobs so that the output of one job becomes the input of the next. This approach breaks a complex calculation into simpler, manageable stages.

• The first job may filter, clean or transform raw data.

• A later job may aggregate the intermediate results by a different key.

• A final job may calculate statistics or produce a report.

• Intermediate data formats must be planned carefully so that successive jobs can consume the output correctly.

For example, a web-log analysis pipeline may first clean log records, next group events into user sessions, and finally calculate session statistics. Independent stages can be retried and debugged separately.

job. Each job in the chain takes the output of the previous job as its input, allowing for a series of transformations and aggregations on the data. This composition is essential for tasks like multi-step data processing pipelines, where intermediate results from one phase need further processing in subsequent phases. For example, in a web log analysis scenario, the first MapReduce job might filter and clean the log data, the second job could aggregate the data by user sessions, and the third job might compute statistics on the session data. Effective composition requires careful planning of the intermediate data format and efficient handling of data transfer between jobs. By breaking down complex calculations into a sequence of simpler MapReduce jobs, developers can leverage the power of distributed computing to handle large-scale data processing in a modular and manageable way. This approach also improves fault tolerance, as each stage can be independently retried and debugged.

Eg:

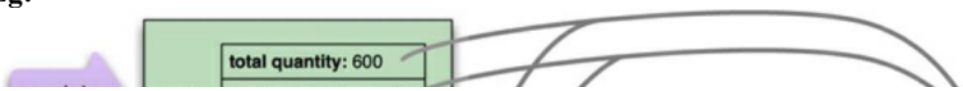


Figure 4. Original illustration of composing MapReduce calculations (PDF page 28).

3.1 Average and count calculations

For an average, each mapper can emit a partial **sum** and **count**. The reducer combines these values by adding the sums and counts. The final average is calculated as:

Average = Total Sum / Total Count

The average of partial averages is generally not correct unless the groups have equal counts. Therefore, combine the sum and count first, and calculate the average from the combined values.

For a simple count, each mapper emits the value 1 for every qualifying record. The reducer sums these values to obtain the total count.

4. A Two-Stage MapReduce Example

A two-stage MapReduce process executes two MapReduce jobs in sequence. The first stage processes raw input and produces intermediate records. The second stage uses those records to perform a further transformation or aggregation. This is useful when a complex calculation cannot be expressed conveniently as one MapReduce job.

The supplied example illustrates creating monthly sales records for a product and then creating base records that support year-on-year comparisons. In the second stage, the mapper prepares records for comparison, and the reduce stage merges incomplete records into consolidated output.

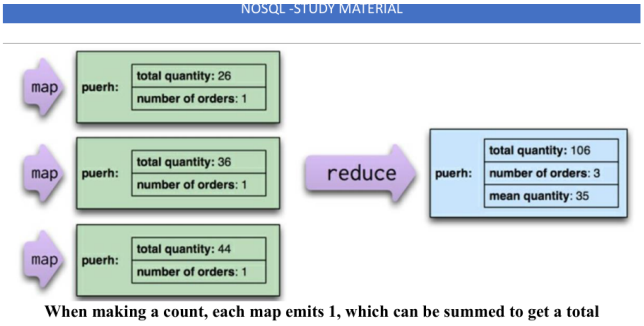


Figure 5(a). Initial two-stage calculation (PDF page 29).

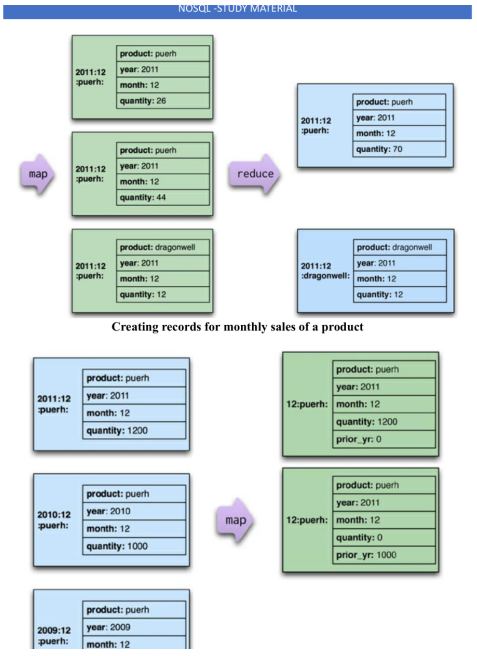


Figure 5(b). Monthly sales records and year-on-year comparison records (PDF page 30).

Key points

- The output of Stage 1 becomes the input of Stage 2.
- Each stage has its own Map and Reduce functions.

• The first stage may produce monthly or intermediate aggregates.

• The second stage may merge records or create comparison-ready summaries.

• Breaking a calculation into stages improves modularity and makes complex processing easier to manage.

5. Incremental MapReduce

Incremental MapReduce is an extension of traditional MapReduce that processes only the changes in a dataset rather than recomputing the entire dataset. It is useful when data is updated continuously, such as in real-time analytics or iterative processing.

• The framework identifies newly added or modified data since the previous computation.

• The Map function processes only the changed input and generates updated intermediate key/value pairs.

• The Reduce function merges the new results with previously computed results.

• This reduces repeated computation, improves resource utilization and can lower processing latency.

The Map stage is particularly suitable for incremental processing because individual map operations are isolated. If an input record has not changed, its mapper does not need to be rerun. Only changed input needs to be processed again.

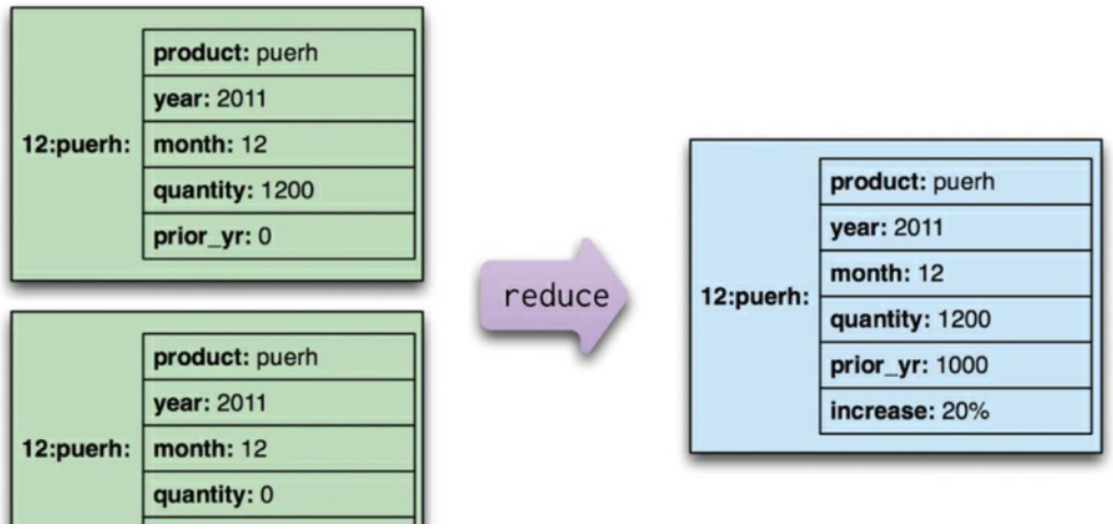


Figure 6. Original incremental MapReduce illustration (PDF page 31).

6. Key–Value Databases

A key–value database is a type of NoSQL database that stores data as a collection of key/value pairs. Each key is unique and is associated with a value. The value may be a simple item such as a string or number, or a more complex object such as JSON or binary data. The database retrieves the value using its key, which enables fast access without complex query processing or table scans.

Examples mentioned in the study material include **Redis, DynamoDB and Riak**. Key–value databases are used in distributed systems and can scale horizontally by partitioning data across nodes, with replication supporting availability and fault tolerance.

6.1 Key–value store terminology

Term	Meaning
Key	Unique identifier used to access a particular value. Keys are commonly strings.
Value	Data associated with a key; may be a string, number, JSON object or binary data.
Bucket / namespace	Logical grouping used to organize and isolate key/value pairs. Some systems use terms such as bucket or table.
Get	Retrieves the value associated with a specified key.
Put	Inserts a new key/value pair or updates the value for an existing key.
Delete	Removes a key/value pair using its key.
TTL (Time to Live)	Defines how long a key/value pair remains valid; the pair is automatically removed after expiry.
Node	An individual server or instance participating in the distributed store.
Cluster	A group of nodes working together as one key–value database.
DHT	Distributed Hash Table; a decentralized structure used to distribute and locate data across nodes.

7. Features of Key–Value Stores

Feature	Explanation
Simplicity	Data is represented as key/value pairs, making the model straightforward and easy to use.
High performance	Get, put and delete operations are optimized for rapid access.
Scalability	Nodes can be added and data partitioned across them to support larger datasets and traffic.
Flexibility	Values can contain different data types, including strings, numbers, JSON and binary data.
Distributed architecture	Replication and partitioning help distribute data and support availability and fault tolerance.
In-memory capability	Some stores, such as Redis, keep data in RAM to improve read and write speed.
Eventual consistency	In distributed deployments, updates may propagate to all nodes over time.
Replication and availability	Copies of data can be maintained across nodes; replication may be synchronous or asynchronous.
Large data volumes	Data can be distributed across nodes to support large-scale applications.
Schema-free model	A fixed schema is not required, so values can evolve without conventional schema migrations.

8. CRUD Operations in Riak

Riak is a distributed NoSQL database designed for high availability, fault tolerance and scalability. CRUD stands for Create, Read, Update and Delete. In a key–value database, these operations work with a bucket, a key and the value stored for that key.

8.1 Create

Create inserts a new object into a bucket under a key. The object contains the value to be stored. The study material introduces connecting to Riak and creating buckets and objects as the first steps in working with Riak.

8.2 Read

Read retrieves an object from Riak using its bucket and key. The application uses the returned value for further processing.

8.3 Update

Update changes the value associated with an existing key. The updated object is written back to the store.

8.4 Delete

Delete removes the object associated with the specified key from the bucket.

8.5 Working with complex objects

A key–value store can keep complex values, such as structured objects, as the value associated with a key. The application accesses the object through the key. The study material includes a separate section on working with complex objects in Riak.

Exam tip: When explaining CRUD, clearly expand each letter and state the purpose of the operation. Mention that the key identifies the object and the bucket provides a logical grouping.

9. Scaling in Riak

Riak is designed for horizontal scaling. A cluster can be expanded by adding nodes, allowing it to handle larger data volumes and increased traffic. Data distribution is managed using **consistent hashing** and **virtual nodes (vnodes)**.

• Each physical node manages multiple virtual nodes.

• Consistent hashing distributes data across the cluster.

• When a node is added, Riak redistributes data to maintain balance.

• Data is replicated across multiple nodes for redundancy; the replication factor can be adjusted according to the required fault tolerance.

• If a node fails, other replicas can continue serving data. Riak can re-replicate data to restore the desired replication factor.

• Riak supports read and write scalability. Read-heavy workloads can read from replicas, while concurrent writes can be handled across nodes.

• Quorum-based reads and writes provide flexibility in balancing consistency and availability.

• Eventual consistency allows updates to propagate to replicas over time; vector clocks help identify and resolve conflicts caused by concurrent writes or network partitions.

These mechanisms make Riak suitable for large-scale distributed applications that require availability and the ability to grow by adding machines.

10. Suitable Use Cases of Key–Value Stores

10.1 Storing session information

A user session can be represented by a unique session ID used as the key. The associated value stores session information such as preferences, authentication tokens or temporary application data. A key–value store provides quick lookup by session ID. In-memory storage supports low latency, and TTL can automatically remove expired sessions.

10.2 User profiles and preferences

A user ID can be used as the key, while the value stores the user profile, settings and preferences in JSON or another structured format. New attributes can be added as requirements change, without enforcing a rigid schema. Fast lookups make this model useful for personalized web and mobile applications.

10.3 Shopping cart data

A shopping cart can be stored under a session ID or user ID. The value contains cart items and related details such as product IDs, quantities, prices, discounts and promotions. Frequent add, remove and update operations can be performed efficiently. TTL can expire inactive carts and help remove stale data.

11. When Not to Use Key–Value Stores

Key–value stores are optimized for direct access using a key. Their simple model is not suitable for every data-processing requirement. The study material identifies the following limitations.

11.1 Relationships among data

Key–value stores are not ideal when an application requires complex relationships such as one-to-many, many-to-many or hierarchical relationships. They do not inherently provide relational joins or foreign-key constraints. For example, representing users and their orders or products and their categories may require additional application-side logic. Relational or graph databases are often better suited to such requirements.

11.2 Multi-operation transactions

Key–value stores are not generally the best choice when multiple operations across several keys must be executed atomically with strong consistency. Examples include transferring money between accounts or updating several related records under complex business rules. Some stores provide limited transaction support, but the model is primarily optimized for individual key/value operations.

11.3 Query by data

A key–value store is designed to retrieve a value using its key. It is less suitable for searching by non-key attributes, such as finding users by age or products within a price range, unless additional indexing or application-side processing is used. Document databases or relational databases provide richer querying facilities for such use cases.

11.4 Operations by sets

Key–value stores are not inherently designed for bulk set operations such as union, intersection and difference across collections of values distributed under multiple keys. Performing such operations may require retrieving and processing many individual pairs, which can be inefficient. Systems with native set or collection support, or graph databases for relationship-centric operations, may be more appropriate.

Requirement	Key–value store suitability	Alternative to consider
Direct lookup by unique key	Very suitable	Key–value store
Session state and cache	Very suitable	Key–value store
Complex relationships and joins	Limited	Relational / graph database
Multi-key ACID transactions	Limited or implementation-dependent	Relational / transaction-capable distributed DB
Rich filtering and range queries	Limited without secondary indexes	Document / relational database
Set operations across collections	Limited	Database with native set/collection support

12. Quick Revision Points

• Map produces intermediate key/value pairs; Reduce combines values having the same key.

• MapReduce workflow: Split → Map → Shuffle/Sort → Reduce → Output.

• Partitioning decides which reducer receives a key.

• Combining performs local aggregation before network transfer.

• A combinable reducer must produce output in a form that can be used as its input.

• Composed MapReduce jobs chain multiple jobs; one job’s output becomes the next job’s input.

• Incremental MapReduce processes only new or modified data.

• A key–value store maps a unique key to a value.

• Core operations: Get, Put and Delete; TTL controls expiry.

• Riak uses consistent hashing, vnodes, replication and quorum-based operations to support scaling and availability.

• Use key–value stores for sessions, profiles, preferences and shopping carts.

• Avoid relying on key–value stores for complex joins, multi-key transactions, rich non-key queries and extensive set operations.

13. Probable VTU Examination Questions

The following are practice questions framed from the topics covered in the supplied Module 3 material. They are **probable/practice questions**, not confirmed or officially released VTU questions.

A. Long-answer questions (8–10 marks)

1. Explain the MapReduce programming model with a neat diagram. Describe the Map and Reduce functions and the complete workflow.

2. Explain partitioning and combining in MapReduce. Discuss the role of a combiner and explain why every reducer is not necessarily combinable.

3. Explain how MapReduce calculations can be composed. Illustrate the calculation of average using sum and count.

4. Explain the two-stage MapReduce example with suitable diagrams, describing the role of each stage.

5. What is Incremental MapReduce? Explain its working and advantages over recomputing the complete dataset.

6. Explain the key–value data model. Discuss key–value store terminology and its important features.

7. Explain CRUD operations in Riak with suitable examples or syntax as covered in the study material.

8. Explain how Riak achieves horizontal scalability. Discuss consistent hashing, virtual nodes, replication, eventual consistency and quorum-based operations.

9. Discuss the suitable use cases of key–value stores with reference to session information, user profiles/preferences and shopping cart data.

10. Explain the situations in which key–value stores should not be used: complex relationships, multi-operation transactions, query by data and set operations.

B. Short-answer questions (3–5 marks)

1. Define MapReduce. State its two main phases.

2. What is an intermediate key/value pair?

3. List and explain the five stages in the MapReduce workflow.

4. Define partitioning. Why is it required?

5. What is a combiner? State its purpose.

6. What is meant by a combinable reducer?

7. Why is the average calculated using sum and count rather than by averaging partial averages?

8. What is a two-stage MapReduce process?

9. Define Incremental MapReduce.

10. Define a key–value database with one example.

11. Explain Get, Put and Delete operations.

12. What is TTL in a key–value store?

13. What are a bucket, node and cluster?

14. What is a Distributed Hash Table (DHT)?

15. What are virtual nodes in Riak?

16. List any four features of key–value stores.

17. Mention three applications of key–value stores.

18. State four situations where key–value stores are not suitable.

C. Application / problem-oriented questions

• For a dataset containing product-wise sales records, identify suitable mapper output keys and values and explain how the reducer can calculate total sales.

• A mapper emits several records with the same key. Explain how shuffle, sort and partitioning ensure that the correct reducer receives them.

• A web application stores each user session using a session ID. Explain how a key–value store and TTL can be used to manage these sessions.

• An application must find all customers in a given age range and join their orders. Justify whether a key–value store is appropriate.

Reference and Scope

Primary study material: *NoSQL – Study Material*, Module 3 Notes, PDF pages 25–37, supplied by the faculty/user. The same study material is available at: https://jnnce.ac.in/jnndemo/aiml/nosql/Notes.pdf.

Scope note: These notes cover only the requested Module 3 pages (25–37). Original diagrams have been reproduced from the supplied PDF and captions identify their source-page locations. Wording has been reorganized for readability and exam preparation while retaining the topics and concepts in the source material.